

From Edge Detection to Facial Segmentation

A Classical Signal Processing Pipeline for Visual Pattern Recognition

Heyang Zhang (Stefan) Zhang

Sichuan University - Pittsburgh Institute

Contents

- Section I: Brackground and Fundamental Theory
- Section II: Edges Detection
- Section III: Pattern Recognition
- Section IV: Facial Semantic Segmentation
- Section V: Demo and Evaluation
- Conclusion

Section I: Background and Fundamental Theory

From convolution in signals to convolution in images

Background

- Convolution first appeared as a core operation in one-dimensional signal processing, where systems were studied through their responses to input signals.
- In this view, a filter slides along a signal and produces a weighted local response, making smoothing, differentiation, and feature measurement mathematically systematic.
- As digital images became common, the same idea was extended from 1D sequences to 2D grids of pixel intensities.
- This historical shift made convolution one of the basic tools for image processing, linking classical signal theory with modern visual analysis.

1D signal processing → convolutional filters → 2D image processing

1D Signal Convolution

In one-dimensional signal processing, convolution describes how an input signal interacts with a system or filter. The output at each location is a weighted sum of nearby input values:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] h[n - k]$$

where:

- $x[k]$ is the input signal,
- $h[n]$ is the filter or impulse response,
- $y[n]$ is the output response.

The main idea is sliding a filter across the signal and computing a local weighted response.

Image as a 2D Signal

- A grayscale image can be modeled as a discrete 2D signal:

$$I[m, n], \quad m = 0, \dots, M-1, \quad n = 0, \dots, N-1$$

- Each pixel stores intensity at a spatial location.
- Edges correspond to local intensity changes in horizontal and vertical directions.
- This lets us reuse the language of systems, responses, and filtering instead of treating images as mysterious AI inputs.

Project image set

Camera



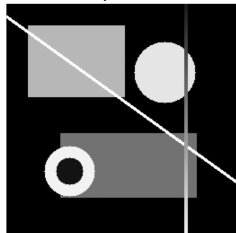
Coins



Astronaut



Synthetic



2D Image Convolution

For an image, the same idea extends from one dimension to two dimensions:

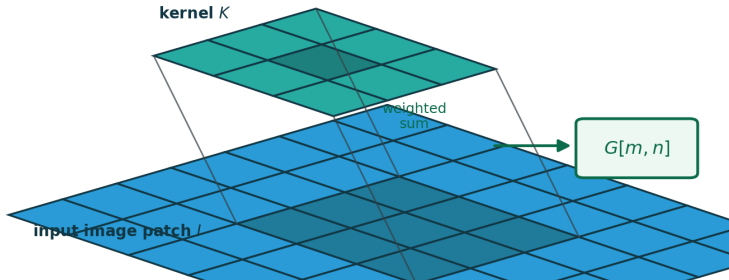
$$G[m, n] = \sum_{i=-a}^a \sum_{j=-b}^b I[m-i, n-j] K[i, j]$$

or more compactly,

$$G = I * K$$

where:

- $I[m, n]$ is the input, $K[i, j]$ is the convolution kernel, and $G[m, n]$ is the output.



Convolution on Images

- A Gaussian kernel performs smoothing by averaging nearby pixels with distance-based weights.
- After smoothing, derivative-based filters respond more clearly to stable boundaries instead of random noise.
- This is why convolution is the foundation of both classical edge detection and later visual feature extraction methods.



(a) Original



(b) Gaussian smoothing



(c) Enhanced detail

Figure: Gaussian convolution smooths a 2D image using distance-weighted local averaging.

Section II: Edges Detection

From image intensity changes to edge-enhanced structure

Edge Detection as Filtering

- Edge detection is a filtering problem based on convolution.
- Derivative-like filters respond strongly to high-frequency structures.
- That is useful for boundaries, but noise also lives in the high-frequency range.
- So the pipeline must balance sensitivity to structure against sensitivity to noise.

Filtering concept for classical edge detection

Gray image as 2D signal



suppress
noise



Gaussian smoothing



highlight
boundaries



Derivative-driven edge response



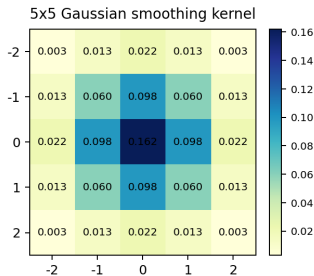
Gaussian Smoothing Kernel

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

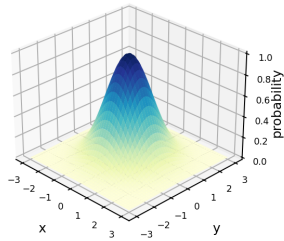
$$K[i, j] = \frac{G_{\sigma}(i, j)}{\sum_{p, q} G_{\sigma}(p, q)}$$

$$I_{\text{smooth}} = I * K$$

- The center pixel receives the largest weight.
- Nearby pixels contribute less as distance increases.
- This suppresses noise before derivative filters amplify edges.



2D Gaussian probability surface



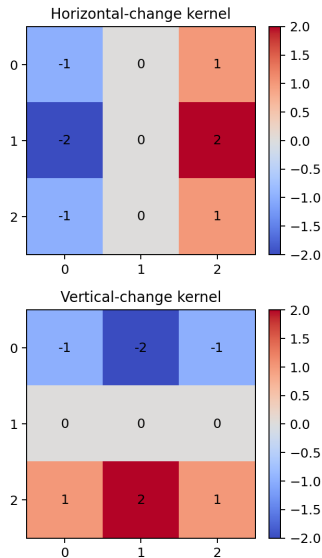
Sobel Operator

$$G_x = I * S_x, \quad G_y = I * S_y$$

$$G = \sqrt{G_x^2 + G_y^2}$$

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- G_x measures horizontal intensity change.
- G_y measures vertical intensity change.
- The gradient magnitude marks boundary strength.
- Sobel is a deterministic convolution filter.



Sobel Responses

The astronaut image is first smoothed with a Gaussian filter

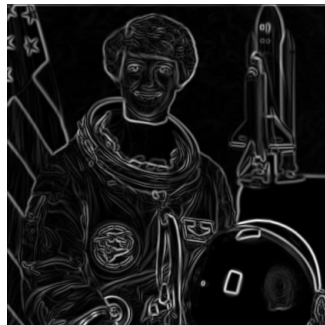
Note: in G_x and G_y , bright regions indicate positive directional change, dark regions indicate negative directional change, and mid-gray means weak response.



(a) G_x response



(b) G_y response



(c) Gradient magnitude G

Canny Edge Detection Pipeline

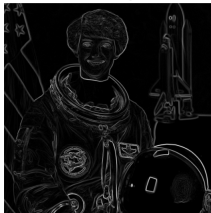
1. Gaussian smoothing to suppress unstable noise.
2. Gradient calculation to estimate local edge strength and direction.
3. Non-maximum suppression to keep thin edge responses.
4. Double thresholding to separate strong and weak candidates.
5. Edge tracking by hysteresis to keep connected meaningful edges.

Astronaut: edge detection in clean and noisy conditions

Original image



Clean Sobel magnitude



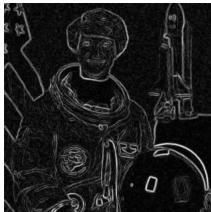
Noisy image



Naive Sobel on noise



Gaussian + Sobel

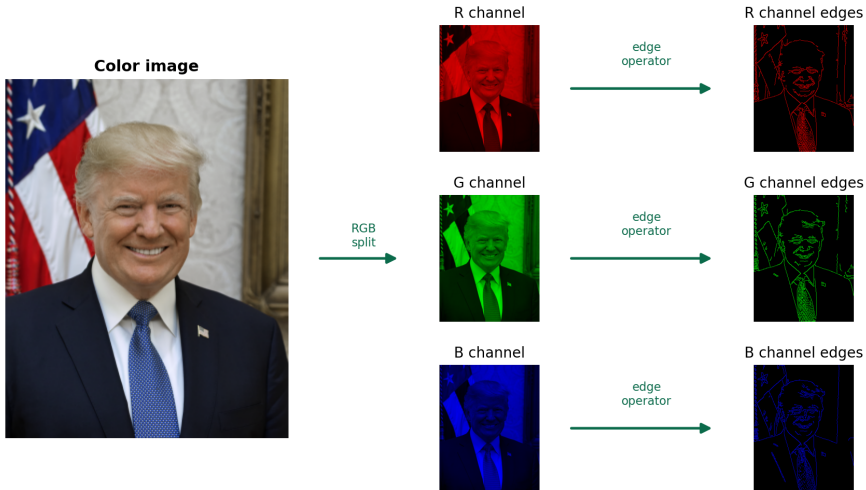


Canny edge map



Color Edge Detection

- A color image can be treated as three 2D signals: R, G, and B.
- The same edge detector can be applied independently to each channel.



Section III: Pattern Recognition

Grouping local edge responses into pattern hypotheses

Why Edges Are Not Enough

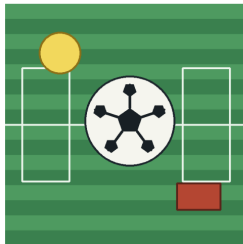
- Edge detection gives local structure, not semantic understanding.
- The output is a set of edge pixels, not labeled parts.
- Pattern recognition requires grouping, geometry, and positional reasoning.
- This is the transition point from pure filtering to classical pattern recognition.

edge pixels $\neq$ object-level pattern understanding

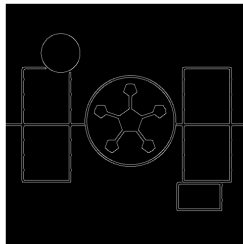
From Edges to Contours

- Neighboring edge pixels can be grouped into connected components.
- Each connected component can be traced as a contour.
- Contours define candidate regions that can be filtered by size, shape, and location.

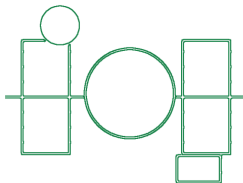
Original image



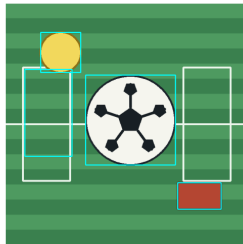
Edge pixels



Connected contour



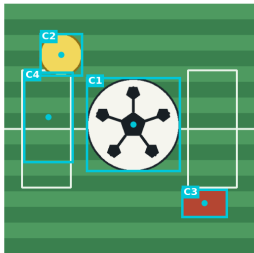
Candidate region



Contour Features

- A contour is still only geometry, not meaning.
- To use it for pattern recognition, we first measure its shape and location.
- Bounding box, area, aspect ratio, centroid, and relative position turn pixels into features.

Measured contour candidates



Feature descriptor

bounding box	(x, y, w, h)
area	$w \times h$
aspect ratio	w/h
centroid	$(x + \frac{w}{2}, y + \frac{h}{2})$
relative position	upper / middle / lower

Pixels become measurable geometric evidence.

From Features to Pattern Rules

- Pattern recognition compares measured features with expected geometric rules.
- No single contour is enough to identify a meaningful pattern.
- A useful pattern usually needs shape, size, and position to agree.

Contour measurements become rule-based hypotheses



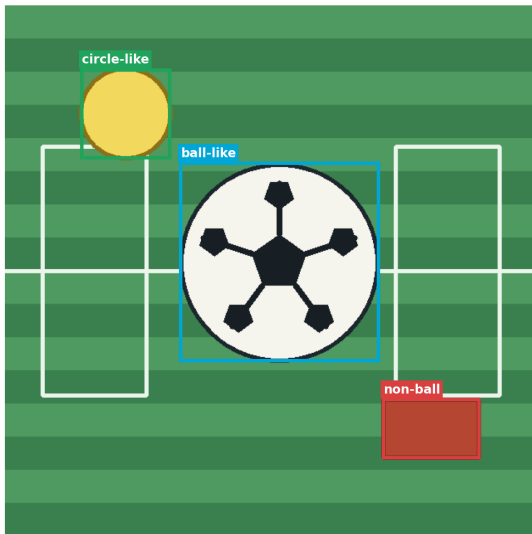
circle-like width $\approx$ height + closed boundary

ball-like large circle + internal texture

non-ball rectangle or weak circular support

Pattern Candidates Are Hypotheses

- Rule output should be treated as a candidate, not a final semantic label.
- False positives are expected because background edges can satisfy simple geometry.
- The next stage checks spatial consistency before interpretation.



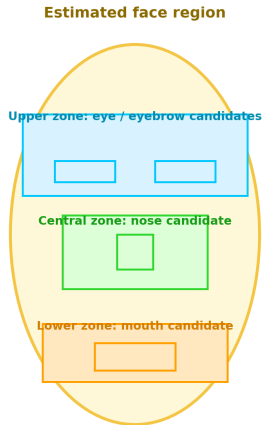
Section IV: Facial Semantic Segmentation

From geometric face-part candidates to semantic regions

Facial Pattern Recognition Logic

- Eyes: small, roughly symmetric horizontal regions in the upper face.
- Nose: a central vertical structure in the middle facial zone.
- Mouth: a wider horizontal contour in the lower face.
- Face outline: the largest oval-like structure or a fallback ellipse.

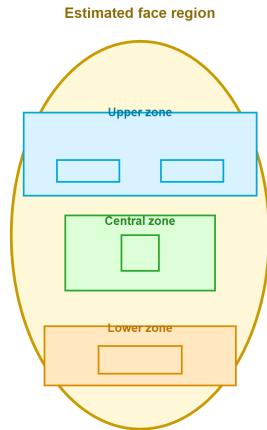
Facial part	Geometric cue
Eyes	symmetry, horizontal aspect ratio, upper-half position
Nose	near-center position, vertical elongation
Mouth	lower-third position, wider contour
Face outline	large central region, oval-like support



Geometric fallback keeps the pipeline deterministic when contour evidence is weak.

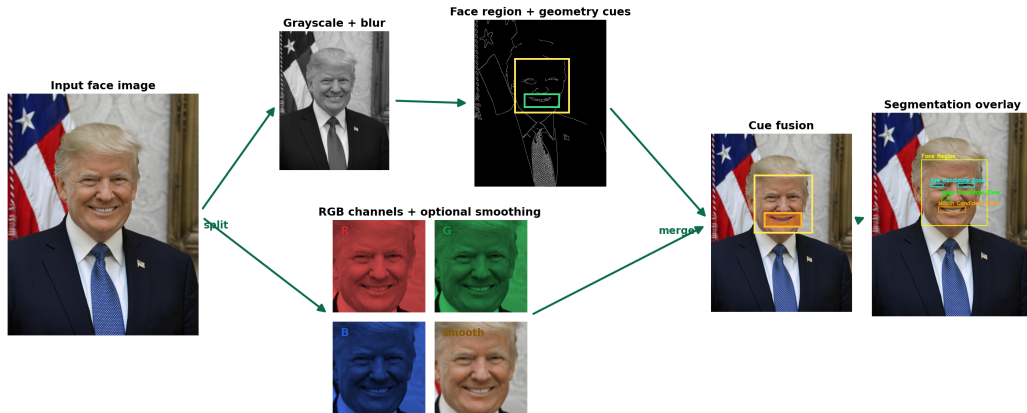
Deterministic Face-Zone Partitioning

- After estimating a broad face region, the algorithm divides it into semantic zones.
- Upper zone: eye and eyebrow candidates.
- Central zone: nose candidate.
- Lower zone: mouth candidate.
- If contour evidence is weak, the system falls back to this deterministic geometry.



Facial Part Segmentation Pipeline

Classical facial-part segmentation pipeline



Section V: Demo and Evaluation

A classroom-friendly Windows application for classical facial-part segmentation

Demo Design

- The user selects a frontal face image from a Windows desktop GUI.
- The application displays original, grayscale, edge map, and segmentation overlay panels.
- The demo saves both intermediate outputs and a combined summary figure.

Select Face Image



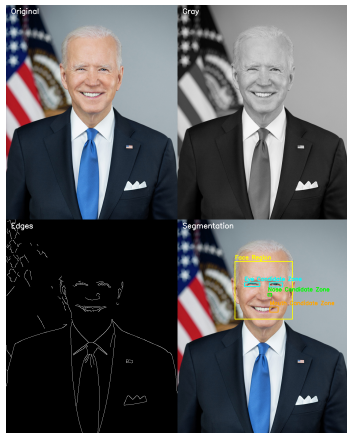
Run Segmentation



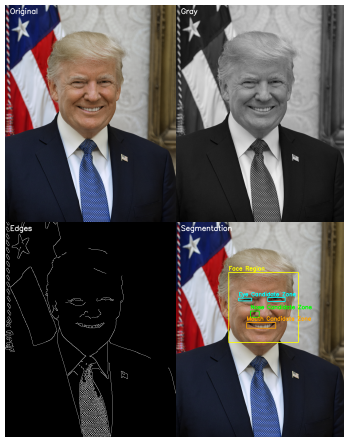
Save Results

Additional Demo Cases

Biden Example



Trump Example



Obama Example

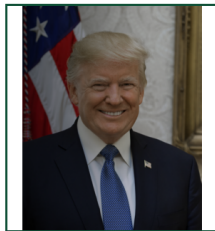


Failure Cases

- Side faces violate the frontal-face assumption.
- Glasses, beard, and hair occlusion disturb contour geometry.
- Strong shadow changes the edge distribution.
- Low resolution weakens local detail.
- Complex backgrounds produce distracting contours.
- Hair crossing the face can create false candidate regions.

Typical failure conditions for a rule-based classical pipeline

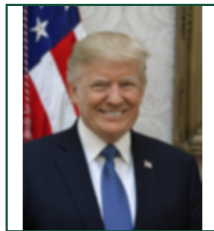
Shadow



Occlusion



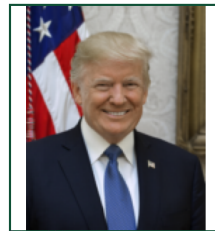
Blur



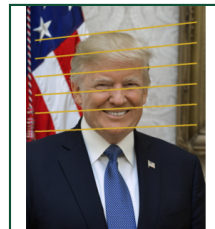
Face in hat



Low resolution



Background clutter



Conclusion

- Edge detection extracts low-level image structure.
- Pattern recognition groups and interprets those structures.
- Facial part segmentation is a natural next step after edge extraction and contour analysis.
- Classical methods are excellent for explanation, but limited in real-world robustness.